COMS 3110: Homework 2
Due: October 10th, 11:59pm
Total Points: 50

Late submission policy. An assignment that is submitted one day late will get a penalty of 10%. An assignment that is submitted two days late will get a penalty of 20%. Assignments submitted after two days will not be graded and will get no points. For example, if an assignment is due on Friday by midnight, a submission on Saturday will be penalized by 10%, and a submission on Sunday will be penalized by 20%. A submission on Monday will get no points.

Submission format. Homework solutions must be **handwritten**. You may either write directly on a tablet or write on paper and scan your work. In both cases, the final submission must be compiled into a **single PDF file**. Typed solutions will only be accepted with prior, explicit permission from the instructor. We reserve the right **NOT** to grade homework that does not follow these requirements. Name your submission file: `<Your-net-id>-3110-hw2.pdf`. For instance, if your netid is `asterix`, then your submission file will be named `asterix-3110-hw2.pdf`. Each student must hand in their own assignment. If you discussed the homework or solutions with others, a list of collaborators must be included with each submission. Each of the collaborators has to write the solutions in their own words (copies are not allowed). Also, recall the AI policy:

- You may **not use AI tools to generate complete solutions** to assignments, coding problems, or any assessed work that is intended to measure your independent knowledge and skills.
- Whenever you use AI tools to assist in your assignment, you must **include a paragraph at the end of the assignment explaining what you used the AI for and include all the prompts you used to get the results. Failure to do so is in violation of the academic honesty policies**.

General Requirements

- When proofs are required, do your best to make them both clear and rigorous. Even when proofs are not required, you should justify your answers and explain your work.
- When asked to present a construction, you should show the correctness of the construction.

Expectations for Runtime Analysis in Algorithm Problems When a problem explicitly asks for one or more of the following components, these are the expectations for a correct solution.

- **Provide runtime:** You are only required to state the final runtime of the algorithm, and full credit will be awarded if your answer is correct. However, if your answer is incorrect, no partial credit will be given unless you also provide a derivation.
- **Provide runtime and derivation:** Show step-by-step reasoning, including the number of iterations of loops and the runtime of each line of pseudocode, leading to the total runtime.
- **Provide exact number of iterations:** Specify the precise number of times each loop executes for the given input size.
- **Provide upper bound on the number of iterations:** State a guaranteed maximum number of iterations for each loop; this may be larger than the exact count.
- **Provide asymptotic bound on the number of iterations:** Describe the growth rate of number of iterations using big-O notation.

Some Useful Equalities

- $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$
- $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$
- $2^{\log_2 n} = n$, $a^{\log_b n} = n^{\log_b a}$, $n^{n/2} \leq n! \leq n^n$, $\log x^a = a \log x$
- $\log(a \times b) = \log a + \log b$, $\log(a/b) = \log a - \log b$
- $a + ar + ar^2 + ... + ar^{n-1} = \frac{a(r^n-1)}{r-1}$
- $1 + \frac{1}{2} + \frac{1}{2^2} + ... + \frac{1}{2^n} = 2(1 - \frac{1}{2^{n+1}})$
- $1 + 2 + 4 + ... + 2^n = 2^{n+1} - 1$

1. **(10 pts)** Given a dynamic array `A` as input, our goal is to construct a heap `H`, which is also stored as a dynamic array.

   Your task is to analyze the time complexity of two different algorithms for building `H`. In each algorithm, the variable `n` represents the number of elements in `A`, i.e., `n = A.length`.

   Note: For both parts (a) and (b), you need to explicitly write down the summation that represents the total runtime of each loop. You may cite the result from class for the final evaluated value of the summation.

   (a) **(5 pts)** Determine the runtime of `BuildHeap1` below and provide the derivation of the runtime. For each line in the pseudocode, clearly specify its **asymptotic (Big-O) runtime bound**. Then, express the **total runtime of the loop** as a summation that accounts for the runtime of each iteration, before evaluating the summation to determine the overall time complexity.

```
BuildHeap1(A){
  Initialize empty array H
  for i = 1 to n {
    H[i] = A[i]
    Heapify-Up(H, i)
  }
}
```

   Initializing an empty array `H` takes constant time, $O(1)$.

   Setting up the for loop takes constant time, $O(1)$.

   For each iteration i of the loop:

   - Assigning `H[i] = A[i]` takes constant time, $O(1)$.
   - Calling `Heapify-Up(H, i)` takes $O(\log i)$ time.

   Thus, the runtime for each iteration i is at most $c_1 \log i + c_2$ for some constants $c_1, c_2 > 0$. The total time for the loop (excluding the constant setup time) is, therefore, bounded by

$$\begin{aligned} \sum_{i=1}^{n} (c_1 \log(i) + c_2) &\leq \sum_{i=1}^{n} (c_1 \log(n) + c_2) \\ &= c_1 n \log n + c_2 n \\ &\in O(n \log n) \end{aligned}$$

   Thus, the total runtime of the algorithm is $O(n \log n)$.

(b) **(5 pts)** Perform the same runtime analysis as in part (a) for `BuildHeap2` below. Clearly specify the **asymptotic (Big-O) runtime bound** for each line of the pseudocode and express the **total runtime of each loop** as a summation that accounts for the runtime of each iteration, before evaluating the summation to determine the overall time complexity.

```
BuildHeap2(A) {
  Initialize empty array H
  for i = 1 to n {
    H[i] = A[i]
  }
  for i = floor(n/2) down to 1 {
    Heapify-Down(H, i)
  }
}
```

Initializing an empty array `H` takes constant time, $O(1)$.
Setting up the first for loop takes constant time, $O(1)$.
For each iteration i of the first loop, assigning `H[i] = A[i]` takes constant time, $O(1)$. Thus, the runtime for each iteration i is at most c_1 for some constant $c_1 > 0$. The total time for the loop (excluding the constant setup time) is, therefore, bounded by

$$\sum_{i=1}^{n} c_1 = nc_1 \in O(n).$$

Setting up the second for loop takes constant time, $O(1)$.
For each iteration i of the second loop, calling `Heapify-Down(H, i)`, takes $O(h)$ time where h is the height of the ith node. As discussed in lecture, an n-element heap has height $\lfloor \log n \rfloor$ and at most $\left\lceil \frac{n}{2^{h+1}} \right\rceil$ nodes of any height h. The total time for the loop (excluding the constant setup time) is, therefore, bounded by

$$\begin{aligned} \sum_{h=0}^{\lfloor \log n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil c_2 h &= c_2 \sum_{h=0}^{\lfloor \log n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil h \\ &\leq 2c_2 n \in O(n) \end{aligned}$$

for some constant $c_2 > 0$. The last inequality follows from the result discussed in lecture:

$$\sum_{h=0}^{\lfloor \log n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil h \leq 2n$$

The total time for the loop is, therefore, $O(n)$.
Thus, the total runtime of the algorithm is $O(n)$.

2. **(20 pts)** You are given a dynamic array A of n integers. The unique elements of A are those that appear only once in A.

   (a) **(10 pts)** Design an efficient algorithm to return a new dynamic array that contains **all and only the unique elements of** A.

   Your score will be evaluated based on the efficiency of your algorithm. More efficient algorithms will receive higher scores.

   Provide the **pseudocode** for your algorithm as well as its **runtime complexity**. Explicitly state whether the runtime is **expected** or **worst-case**.

   Idea: Use 2 hash tables, `H_seen` and `H_dup`, to keep track of elements that have been seen at least once and elements that have been seen more than once, respectively. Then, create a dynamic array `uniq`, go through each element of A and add one that is not in `H_dup` to `uniq`.

   Algorithm:

```
Set up hash tables H_seen and H_dup
for i = 1 to n {
    if (A[i] is present in H_seen) {
        if (A[i] is not present in H_dup)
            insert A[i] to H_dup
    }
    else
        insert A[i] to H_seen
}

Set up an empty dynamic array uniq
for i = 1 to n {
    if (A[i] is not present in H_dup)
        insert A[i] to uniq
}

return uniq
```

   Expected Runtime Analysis: Each hash operation (checking whether `A[i]` is in `H_seen` or `H_dup` and inserting `A[i]` to `H_seen` or `H_dup`) takes $O(1)$ expected time. Since there are n iterations, each iteration takes $O(1)$ expected time, the total expected runtime for the first loop is $O(n)$.

   Setting up an empty dynamic array takes constant time. The second for loop runs n times, each iteration taking $O(1)$ expected time, making the total expected runtime for the second for loop $O(n)$.

   So the total expected runtime is $O(n)$.

(b) **(10 pts)** Now suppose you are also given an integer k, along with the same dynamic array A of n integers.

Design an efficient algorithm to return a dynamic array that contains the k **smallest unique elements** of A. If fewer than k unique elements exist, return all of them.

Your score will be evaluated based on the efficiency of your algorithm. More efficient algorithms will receive higher scores.

Provide the **pseudocode** for your algorithm as well as its **runtime complexity**. Explicitly state whether the runtime is **expected** or **worst-case**.

First, we use the first `for` loop from part (a) to construct `H_seen` and `H_dup`.

Then, use a max-heap H to maintain the k smallest unique elements similar to the problem of finding the kth largest integer in an array covered in lecture.

```
Set up an empty max-heap H as a dynamic array

for i = 1 to n {
    if (A[i] is not present in H_dup) {
        if (H.size() < k)
            insert A[i] to H
        else if (A[i] < H.max())
            remove the maximum element of H
            insert A[i] to H
}

return H
```

For each iteration, checking whether `A[i]` is in `H_dup` takes $O(1)$ expected time. Checking the size of `H` and computing `H.max()` takes constant time. Removing the maximum element of `H` and inserting `A[i]` to `H` takes $O(\log k)$ time. Since the for loop runs n times, the total expected runtime is $O(n \log k)$.

3. **(10 pts)** Given an array A of n distinct integers and a target integer k, write an efficient algorithm to determine whether there exist 2 distinct elements x, y such that $|x-y| = k$. Your algorithm should have **expected** runtime of $O(n)$ or better. Provide the **expected** runtime of your algorithm.

To determine whether there exist two distinct elements x and y such that $|x - y| = k$, we observe that this condition is equivalent to either of the following:

- $x - y = k$, which can be rewritten as $x - k = y$.
- $y - x = k$, which can be rewritten as $x + k = y$.

We use a hash table T to store elements of A encountered so far. Then, we iterate through all elements $A[i]$ and compute $A[i]+k$ and $A[i]-k$. If either of them is already in T, it means that there exists a previous element $A[j]$, where $j < i$, such that either $A[i] + k = A[j]$ or $A[i] - k = A[j]$, satisfying the condition. Otherwise, $A[i]$ is added to T and the process continues.

```
Setup a hash table T
for i = 1 to n {
  if ( (A[i] + k) or (A[i] - k) is present in hash table T)
    return true
  else
    insert A[i] to hash table T
}
return false
```

The loop runs $O(n)$ times. Computing $A[i] + k$ and $(A[i] - k)$ takes constant time, $O(1)$. The expected time for checking whether they are in T and adding $A[i]$ to T is $O(1)$. So, the total expected runtime is $O(n)$.

4. **(10 pts)** Given **sorted** arrays `L[1]`, `L[2]`, ..., `L[K]`, where $K \geq 1$. Each array `L[i]`, `i` $\in \{1, \ldots, K\}$ contains $N \geq 1$ elements. Write an **efficient** algorithm to merge `L[1]`, `L[2]`, ..., `L[K]` into a single sorted array. Give the runtime of your algorithm **as well as its derivation**.

   Your score will be evaluated based on the efficiency of your algorithm. More efficient algorithms will receive higher scores.

   Hint: A naive brute-force approach that repeatedly selects the smallest available element from the arrays results in a runtime of $O(NK^2)$. Consider using a heap, exactly like what we did in recitation, to achieve a better runtime.

   The main idea is to use min heap `H` to extract the smallest element among the heads of all arrays. Each node in `H` is a triple `(val, i, j)` where `val` is the value of the node, `i` is the list index, and `j` is the element index, i.e., `L[i][j] = val`.

```
// Step 1: Insert the first element of each list into the heap
H = []  // minheap
for i = 1 to K {
  H.push(L[i][0], i, 0)
}

result = []

// Step 2: Process the heap until it's empty
for iter = 1 to KN {
  (val, i, j) = H.pop()
  append val to result

  // Step 3: Push the next element from the same list into the heap
  if (j < N) {
    H.push(L[i][j+1], i, j+1)
  }
}
return result
```

   Step 1: As discussed in lecture, `H.push(L[i][0], i, 0)` takes $O(\log(K))$ time. The for loop runs K times. So the total time for Step 1 is $O(K \log(K))$.

   Step 2 and 3: The for loop runs KN times. Each iteration involves:

   - `H.pop()` takes $O(\log(K))$ since extracting the smallest element in a minheap requires heap restructuring.
   - `append val to result` takes $O(1)$.
   - Inserting the next element into the heap takes $O(\log(K))$.

So the total time for Step 2 and 3 is $O(KN\log(K))$.

Since $O(KN\log(K))$ dominates $O(K\log(K))$, the total time complexity is $O(KN\log(K))$.